CSA4707– DEEP LEARNING

ASSIGNMENT – 1

Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System

CO1: Mathematical foundations of deep learning

Student Name	**A.Sai Joshitha**
Roll Number	**192424220**
Department / Year	**AI&DS / 3rd year**
Submission Date	**12-10-2026**

Assignment Overview

AgriVision is an agricultural technology system intended to screen smartphone photographs of crop leaves and identify disease categories. This assignment studies how image representation, linear algebra, probability, numerical computation, learning algorithms, model capacity and optimisation affect the reliability and practical deployment of the classifier.

The initial study uses a public leaf-disease dataset, such as the tomato subset of PlantVillage, with Healthy, Early Blight and Late Blight classes. Images may be resized to 64 × 64 grayscale, giving 4,096 pixel features per image. The final design challenge expands the requirement to 38 crop–disease classes, approximately 20% noisy labels, inference within 50 ms per image and model storage within 5 MB.

Experimental Protocol

- Record the dataset source, selected classes, number of images per class and any exclusions.
- Use reproducible random seeds and document every preprocessing step.
- Separate training, validation and test data. Fit transformations only on training data.
- Choose hyperparameters with validation data or cross-validation; reserve the test set for final evaluation.
- Report measured results from Google Colab. Do not replace missing results with estimates or fabricated values.
- For timing and model size, record the measurement method and environment. Colab timing alone does not prove smartphone compliance. **Contents**

- Task 1 – Problem Structure and Data Representation

- Task 2 – Feature-Space Analysis Using Linear Algebra
- Task 3 – Probability and Information Theory
- Task 4 – Numerical Computation and Stability
- Task 5 – Comparison of Baseline Learning Strategies
- Task 6 – Capacity, Overfitting and Underfitting
- Task 7 – Bias–Variance Decomposition and Estimation
- Task 8 – Gradient-Based Optimisation
- Task 9 – Experimental Performance Across Dataset Sizes
- Task 10 – Theory Versus Experimental Behaviour
- Task 11 – Whether Deep Learning Is Justified
- Task 12 – Design Challenge: Changed Requirements

Task 1 – Analyse the Problem Structure and Data Representation

Objective: Decompose AgriVision into its learning-task components and examine how representation and preprocessing affect the information available to the classifier.

1.1 Learning-task components

Component	Description
Input	A smartphone photograph of a crop leaf.
Output	A predicted class: Healthy, Early Blight or Late Blight.
Learning task	Supervised multiclass image classification using labelled examples.
Performance measure	Accuracy, macro precision, macro recall, macro F1-score and the confusion matrix; class-wise recall is important when disease cases must not be missed.

1.2 Data structures and shapes

- A pixel is a scalar intensity value. A grayscale image is a 64 × 64 matrix.
- A flattened image is a vector $x \in R^{4096}$. Flattening makes it compatible with classical models but removes the explicit two-dimensional neighbourhood structure.

- A batch of 32 grayscale images has shape (32, 64, 64); the corresponding flattened batch has shape (32, 4096).
- A dataset of N flattened images is a matrix $X \in R^{(N \times 4096)}$. Labels are a vector y of length N; onehot labels can be represented as an $N \times C$ matrix for C classes.
- A mini-batch limits the amount of data processed in one optimisation update and affects memory use and gradient noise.

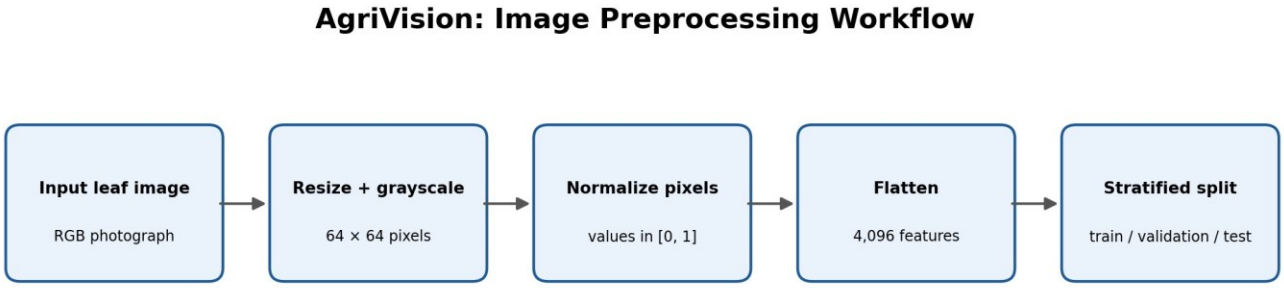
1.3 Norms of a feature vector

For a feature vector $x = (x_1, \ldots, x_n)$, the L^1 norm is $\|x\|_1 = \Sigma_i |x_i|$, the L^2 norm is $\|x\|_2 = \sqrt{(\Sigma_i x_i^2)}$, and the $L\infty$ norm is $\|x\|\infty = \max_i |x_i|$. L^1 measures total absolute magnitude, L^2 measures Euclidean magnitude, and $L\infty$ identifies the largest individual feature value.

1.4 Preprocessing consequences

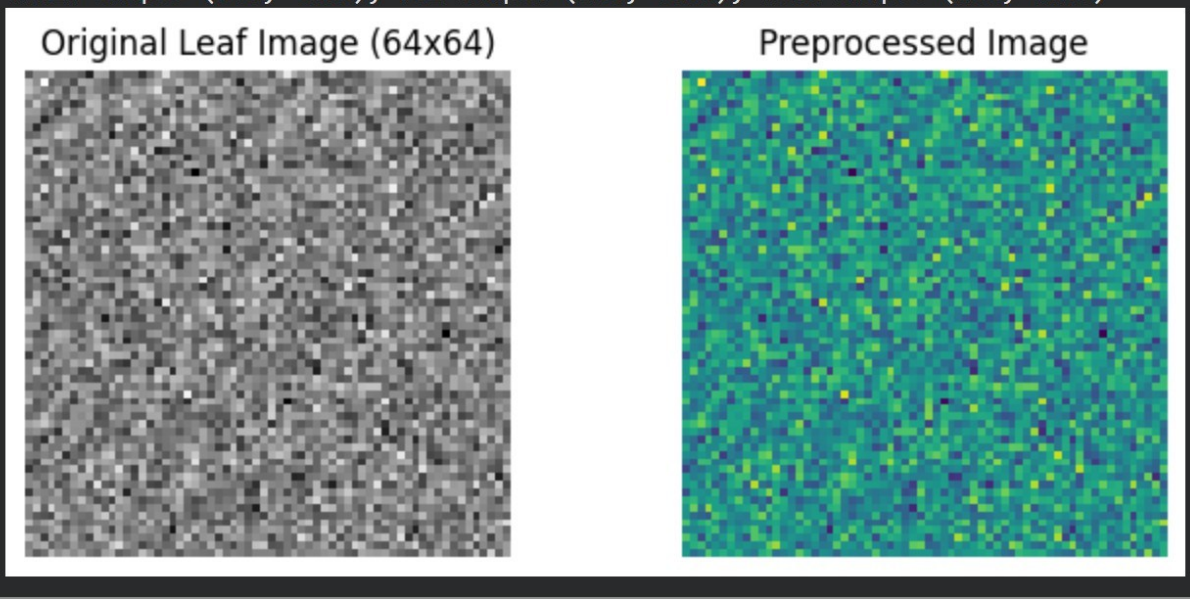
- Resizing to 64×64 reduces computation and storage, but fine disease patterns may disappear.
- Grayscale conversion reduces three colour channels to one. It lowers dimensionality but can remove useful colour cues associated with disease.
- Normalisation to [0,1] puts pixel values on a consistent scale and can improve numerical behaviour during optimisation.
- Flattening creates 4,096 features but discards the explicit spatial arrangement that convolutional models can exploit.
- A stratified train/validation/test split supports fair evaluation while preserving class proportions. Duplicate or near-duplicate images across splits can cause leakage and overly optimistic results.

Record the actual class counts, shapes, norm values and split sizes from the executed notebook. These measurements determine the dataset-specific interpretation.



Output

```
--- TASK 1: PROBLEM STRUCTURE AND DATA REPRESENTATION ---
Class 0: Count = 400, Proportion = 33.33%
Class 1: Count = 400, Proportion = 33.33%
Class 2: Count = 400, Proportion = 33.33%
Feature vector norms -> L1: 1321.95, L2: 20.90, L-infinity: 0.49
Train shape: (840, 4096), Val shape: (180, 4096), Test shape: (180, 4096)
```



Task 2 – Examine the Structure of the Feature Space Using Linear Algebra

Objective: Use eigendecomposition and/or singular value decomposition to determine how information is distributed across the 4,096-dimensional image representation.

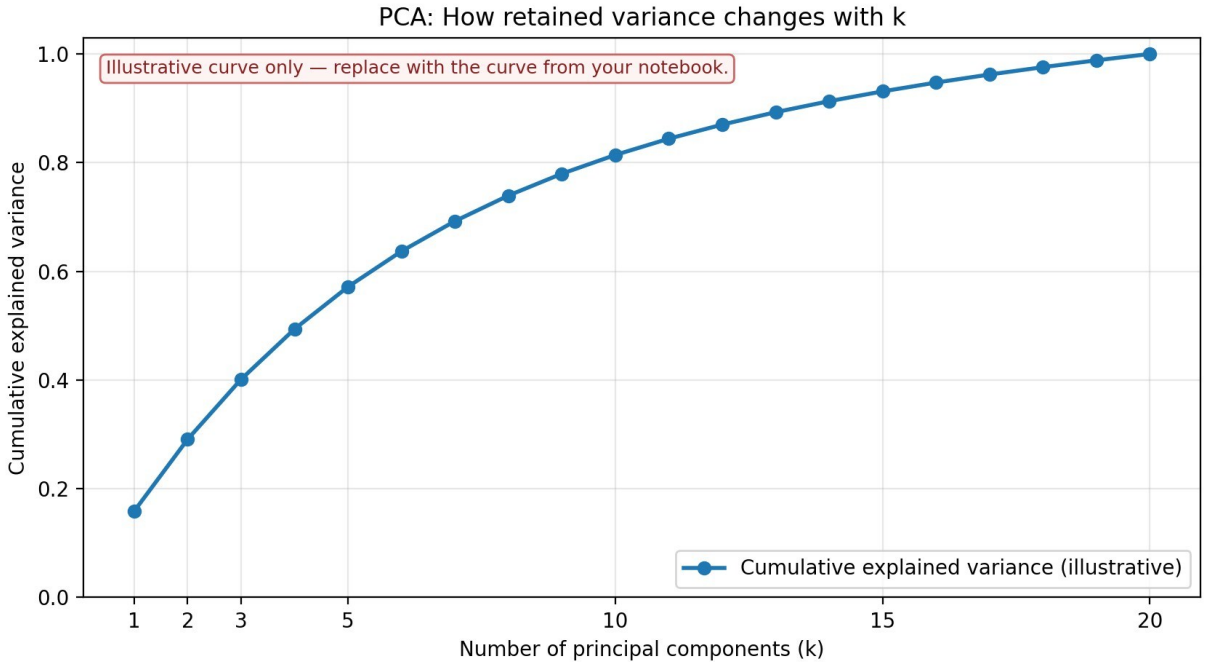
2.1 PCA and SVD

Let X be the training feature matrix. After centring each feature, the covariance matrix is $S = X^TX/(N-1)$. Its eigenvectors identify principal directions and its eigenvalues quantify variance along those directions. In SVD, $X = U\Sigma V^T$; the singular values σ_i are related to covariance eigenvalues by $\lambda_i = \sigma_i^2/(N-1)$ for the centred matrix.

2.2 Explained variance and reconstruction

The explained-variance ratio for component i is $\lambda_i / \Sigma_j \lambda_j$. The cumulative explained variance for the first k components is $\Sigma_{i=1}^k \lambda_i / \Sigma_j \lambda_j$. A rank-k PCA reconstruction projects each image onto the first k components and reconstructs it in the original feature space. Reconstruction error can be measured using mean squared error or the Frobenius norm.

As k increases, reconstruction error should not increase when using the same fitted PCA basis. A practical compression point balances the retained variance and reconstruction quality against computational cost and classification performance.



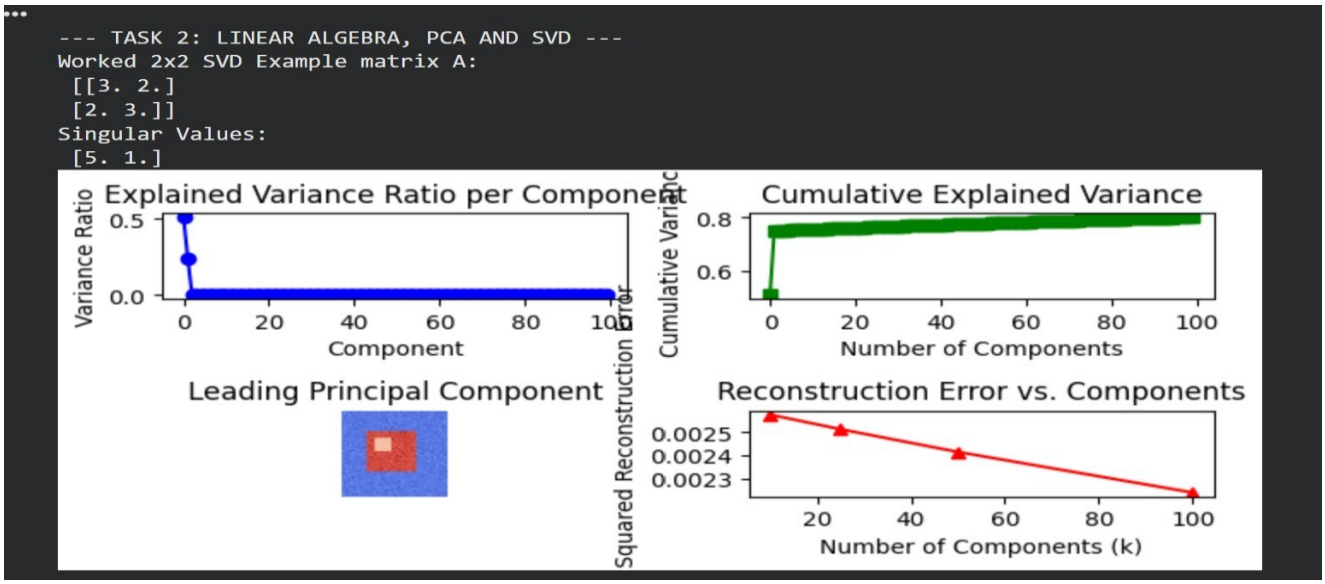
2.3 Interpreting components and cost

- Leading principal components may capture broad brightness, leaf shape, background variation or repeated texture patterns; visualise them as 64 × 64 images before assigning meaning.
- A steeply decreasing spectrum suggests that much of the variance can be represented by fewer directions. A flatter spectrum suggests that information is more widely distributed.
- Exact full SVD can be expensive for large matrices. Randomised or truncated SVD can reduce computation when only leading components are required.
- Variance retained is not identical to disease information retained. A component with modest variance may still contain a useful disease cue, so compare classification performance after compression.

2.4 Small worked example

For a small matrix A, compute A^TA, solve $\det(A^TA - \lambda I) = 0$ to obtain eigenvalues, and find an eigenvector for each eigenvalue. Singular values are $\sigma_i = \sqrt{\lambda_i}$. Normalise the eigenvectors to form V; obtain U from Av_i/σ_i when $\sigma_i \neq 0$. Use the actual numerical matrix and intermediate values generated in the notebook.

Output



Task-03 Analyse Uncertainty Using Probability and Information Theory

Objective: Examine uncertainty in the class labels and model predictions using entropy, cross-entropy and KL divergence.

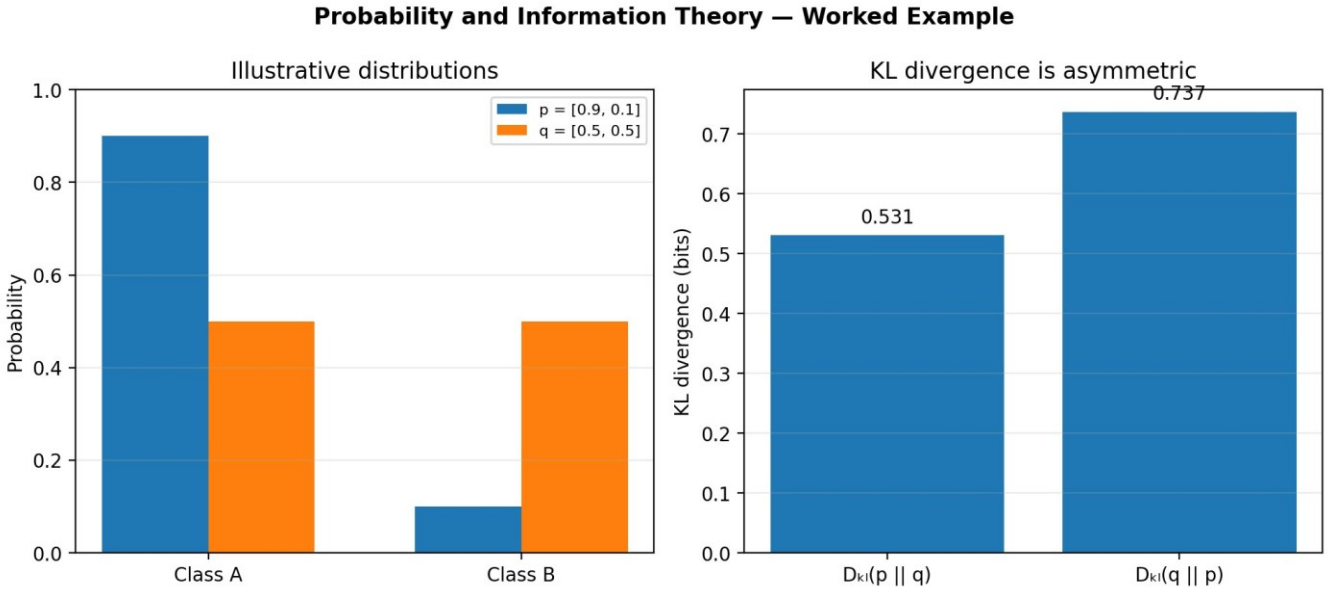
3.1 Class entropy

If class c has probability p(c), the class entropy in bits is $H(p) = -\Sigma c\ p(c) \log_2 p(c)$. Entropy is low when one class dominates and higher when class probabilities are more evenly distributed. Class imbalance can

cause a classifier to favour common classes, so overall accuracy should be accompanied by class-wise and macro-averaged metrics.

3.2 Cross-entropy and KL divergence

For true distribution p and predicted distribution q, cross-entropy is $H(p,q) = -\Sigma_c\, p(c) \log_2 q(c)$. KL divergence is $D_{KL}(p\|q) = \Sigma_c\, p(c) \log_2[p(c)/q(c)]$. They satisfy $H(p,q) = H(p) + D_{KL}(p\|q)$. KL divergence is non-negative and equals zero only when the distributions match wherever p has positive probability.



3.3 Asymmetry and classifier interpretation

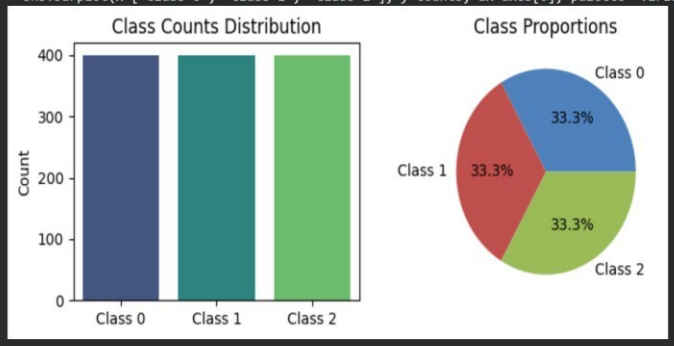
KL divergence is asymmetric in general: $D_{KL}(p\|q)$ need not equal $D_{KL}(q\|p)$. The first direction weights discrepancies according to p; the reverse direction weights them according to q. Use explicit numerical distributions to demonstrate the difference. For model comparisons, calculate cross-entropy and KL divergence from actual predicted probabilities, using safe clipping only to avoid log(0) and documenting the numerical safeguard.

Output

```
--- TASK 3: PROBABILITY AND INFORMATION THEORY ---
Empirical Class Entropy: 1.5850 bits
/tmp/ipykernel_1416/2902298694.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effec

  sns.barplot(x=['Class 0', 'Class 1', 'Class 2'], y=counts, ax=axes[0], palette='viridis')
```



Task-04 Diagnose Numerical Computation Issues

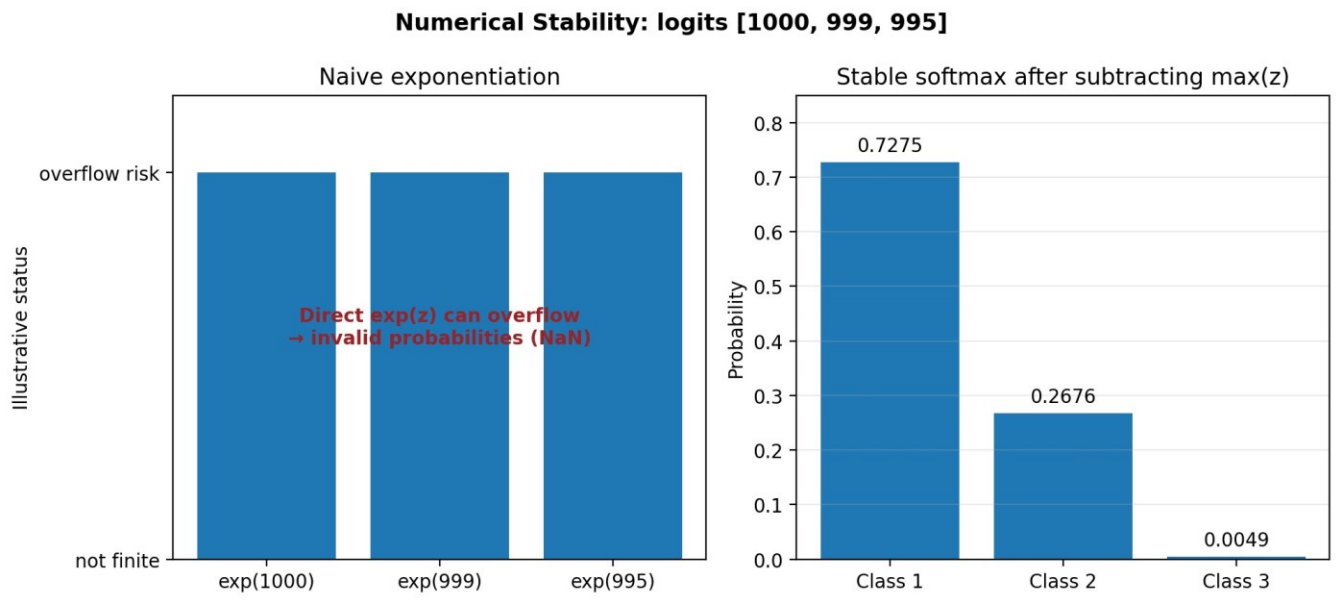
Objective: Demonstrate numerical failures in probability calculations and relate feature conditioning to optimisation behaviour.

4.1 Softmax overflow

Softmax converts logits z_i to probabilities $p_i = \exp(z_i)/\Sigma_j \exp(z_j)$. Large positive logits can make $\exp(z_i)$ overflow; very negative logits can underflow toward zero. For z = [1000, 999, 995], directly exponentiating the logits can produce infinities and invalid divisions.

4.2 Stable softmax and log-sum-exp

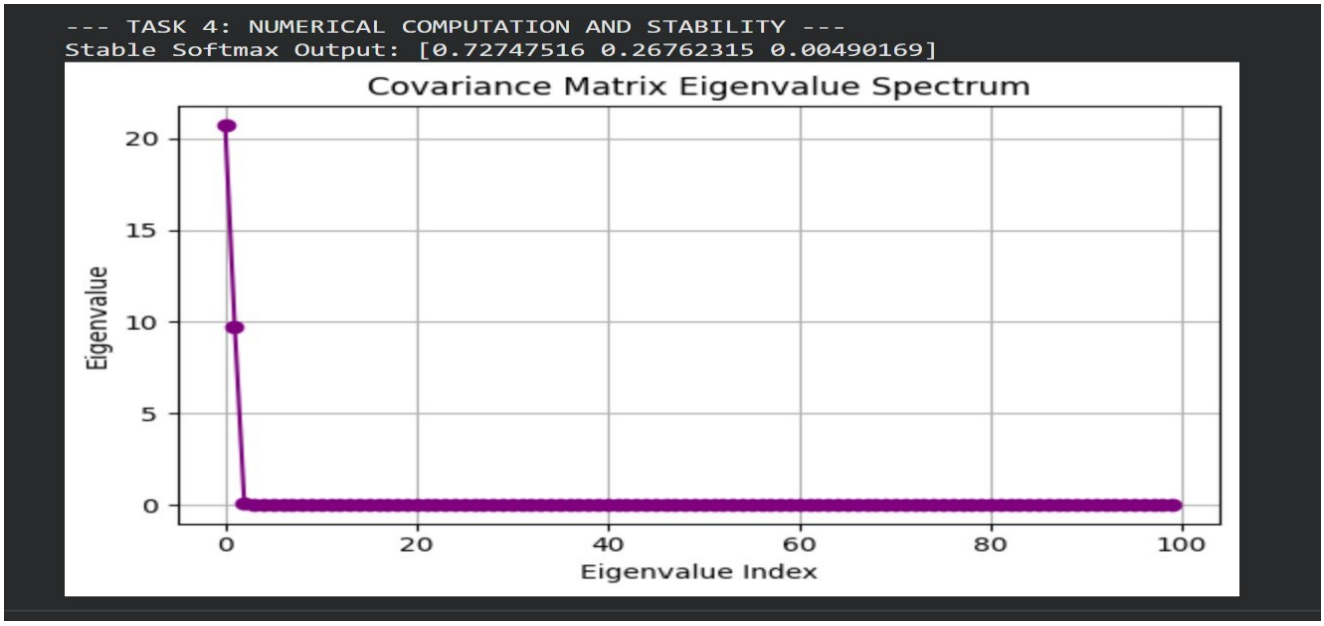
Softmax is unchanged if the same constant is subtracted from every logit. Therefore, compute $\exp(z_i - \max(z))$ and normalise these values. For log-sum-exp, use $\max(z) + \log(\Sigma_i \exp(z_i - \max(z)))$. This preserves the mathematical result while avoiding unnecessarily large exponentials.



4.3 Condition number

For a nonsingular covariance matrix, the condition number in the 2-norm is the largest singular value divided by the smallest singular value; for a symmetric positive-definite covariance matrix, it is λmax/λmin. A large condition number indicates directions with very different scales or curvature. Gradient-based optimisation may zigzag or converge slowly, and the learning rate must be chosen conservatively. Near-zero eigenvalues make the condition number very large or effectively infinite.

Output



Task-05 Differentiate Baseline Learning Strategies

Objective: Compare at least three distinct learning algorithms on raw and PCA-compressed features. **5.1 Models and inductive biases**

Model	Learning strategy	Expected strengths and limitations
Multinomial logistic regression	Parametric linear classifier	Efficient prediction and interpretable linear decision boundaries; may underfit complex leaf patterns.
K-nearest neighbours	Non-parametric, distance-based classifier	Flexible local decisions; prediction can be expensive and high-dimensional distances may become less informative.
Decision tree	Hierarchical feature partitioning	Captures nonlinear interactions and is easy to inspect; deep trees can overfit and become large.

5.2 Raw versus compressed features

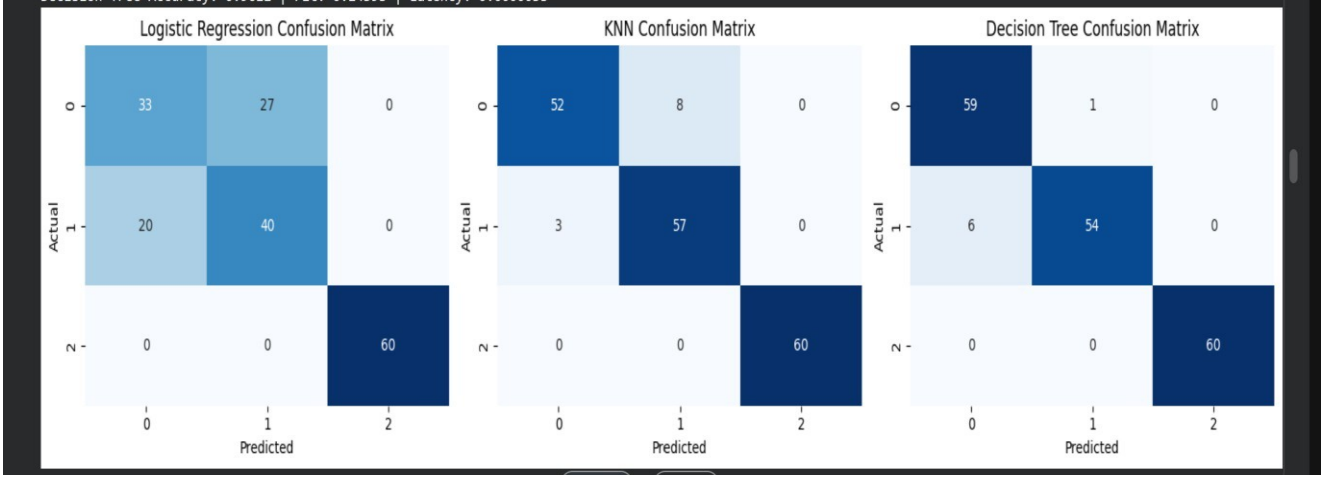
Raw features preserve all resized pixel values but have 4,096 dimensions. PCA reduces dimensionality and may remove redundant variation, reducing storage and computation. However, PCA is unsupervised and preserves variance rather than class separation directly. The effect of compression therefore needs to be measured separately for each classifier.

5.3 Evaluation

Report accuracy, macro precision, macro recall, macro F1-score and confusion matrices. Measure training time, inference latency and serialized model size using a consistent protocol. Fit scaling and PCA only on training data; select hyperparameters on validation data and use the test set only for final evaluation.

Output

```
--- TASK 5: THREE DISTINCT BASELINE CLASSIFIERS ---
Logistic Regression Accuracy: 0.7389 | Fit: 0.2664s | Latency: 0.000003s
KNN Accuracy: 0.9389 | Fit: 0.0012s | Latency: 0.000059s
Decision Tree Accuracy: 0.9611 | Fit: 0.1439s | Latency: 0.000003s
```



Task 6 – Analyse Capacity, Overfitting and Underfitting

Objective: Examine how model capacity affects training error, validation error and generalisation.

6.1 Capacity and error

A model with insufficient capacity may fail to capture disease-related patterns, producing high training and validation error; this is underfitting. As capacity increases, training error often decreases. If the model begins fitting noise or incidental details, validation error may rise, indicating overfitting. The generalisation gap is validation error minus training error.

6.2 Hyperparameter experiments

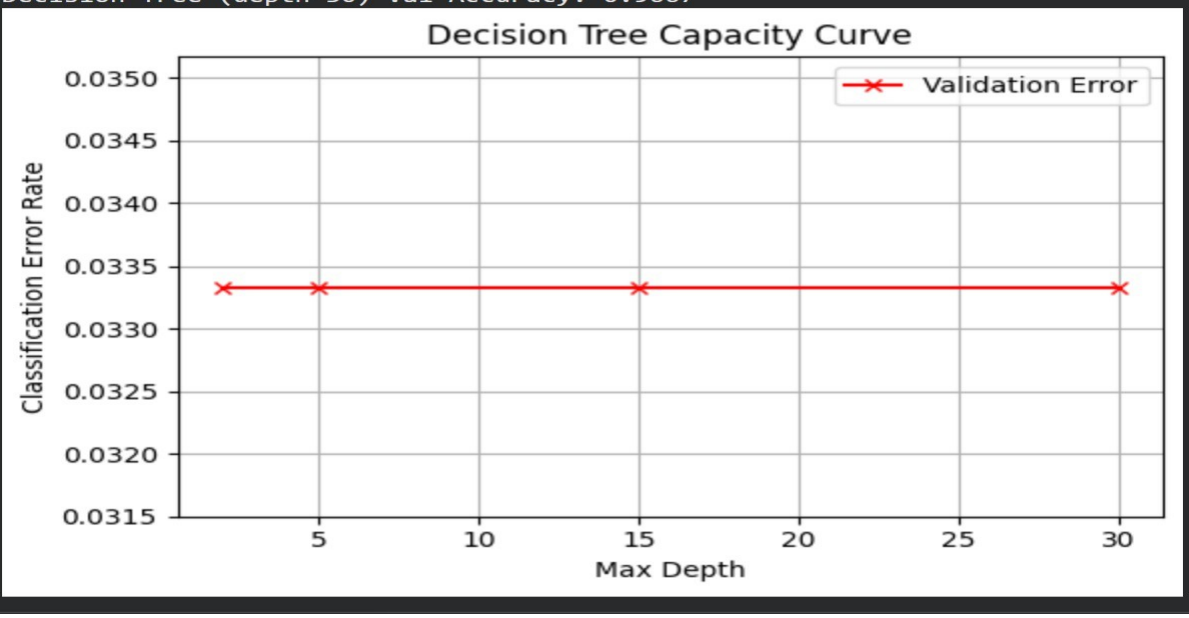
- Decision tree: vary maximum depth or minimum samples per leaf. Deeper trees can model more detailed partitions but may memorise the training set.
- K-nearest neighbours: vary k. Very large k produces smoother, less flexible decisions; very small k is sensitive to individual training examples and noise.
- Logistic regression: vary regularisation strength. Stronger regularisation restricts effective model complexity; weaker regularisation allows a more flexible fit.
- Plot training and validation errors against each hyperparameter and identify the minimum validation error.

6.3 Avoiding test-set bias

Repeatedly selecting hyperparameters using test results leaks information from the test set into model design and produces an optimistically biased final estimate. Use a validation set or cross-validation for model selection, then evaluate the chosen model once on the held-out test set.

Output

```
--- TASK 6: CAPACITY, OVERFITTING AND UNDERFITTING ---
Decision Tree (depth=2) Val Accuracy: 0.6722
Decision Tree (depth=5) Val Accuracy: 0.9000
Decision Tree (depth=15) Val Accuracy: 0.9667
Decision Tree (depth=30) Val Accuracy: 0.9667
```



Task 7 – Decompose Error into Bias and Variance

Objective: Compare low- and high-capacity models using bootstrap resampling and examine maximumlikelihood and maximum-a-posteriori estimation.

7.1 Bootstrap experiment

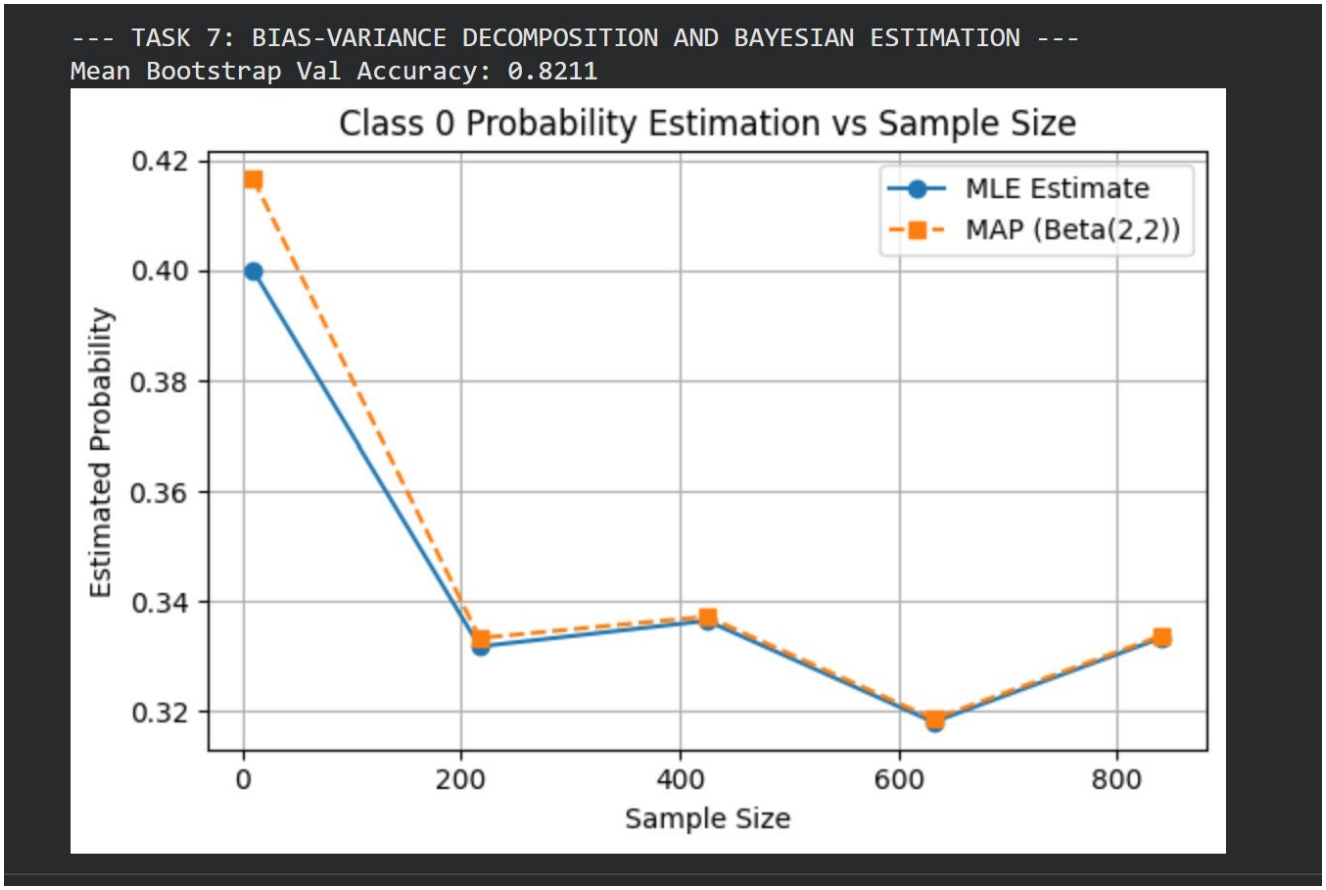
Create at least 10 bootstrap training replicas by sampling training examples with replacement. Fit the selected model on each replica and obtain predictions or class probabilities for the same held-out examples. Compare prediction variability and error for a low-capacity and a high-capacity model.

The classical squared-error decomposition is $E[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \text{irreducible noise}$ under its assumptions. Classification requires a clearly specified decomposition, such as a probability-based Brier-score analysis or a documented approximation. The components should not be presented as an exact regression identity if the chosen classification method does not support it.

7.2 MLE and MAP

For n observations with s successes, the Bernoulli maximum-likelihood estimate is $\hat{p}$ MLE = s/n. With a Beta(α,β) prior, the posterior is Beta(α+s, β+n−s), and the MAP estimate for α+s > 1 and β+n−s > 1 is $\hat{p}$ MAP = (α+s−1)/(α+β+n−2). A prior can have a noticeable influence for small samples; its relative influence generally decreases as sample size increases.

Output



Task 8 – Analyse Gradient-Based Optimisation

Objective: Implement softmax regression from scratch and compare batch gradient descent, SGD and mini-batch SGD.

8.1 Softmax regression

For input matrix X, weight matrix W and bias vector b, logits are Z = XW + b. Row-wise softmax gives probabilities P. With one-hot target matrix Y and N samples, the average cross-entropy loss is $L = -(1/N)\,\Sigma_i \Sigma_c\, Y_{ic} \log(P_{ic})$. The gradients are $\partial L/\partial W = X^T(P-Y)/N$ and $\partial L/\partial b = (1/N)\Sigma_i(P_i - Y_i)$.

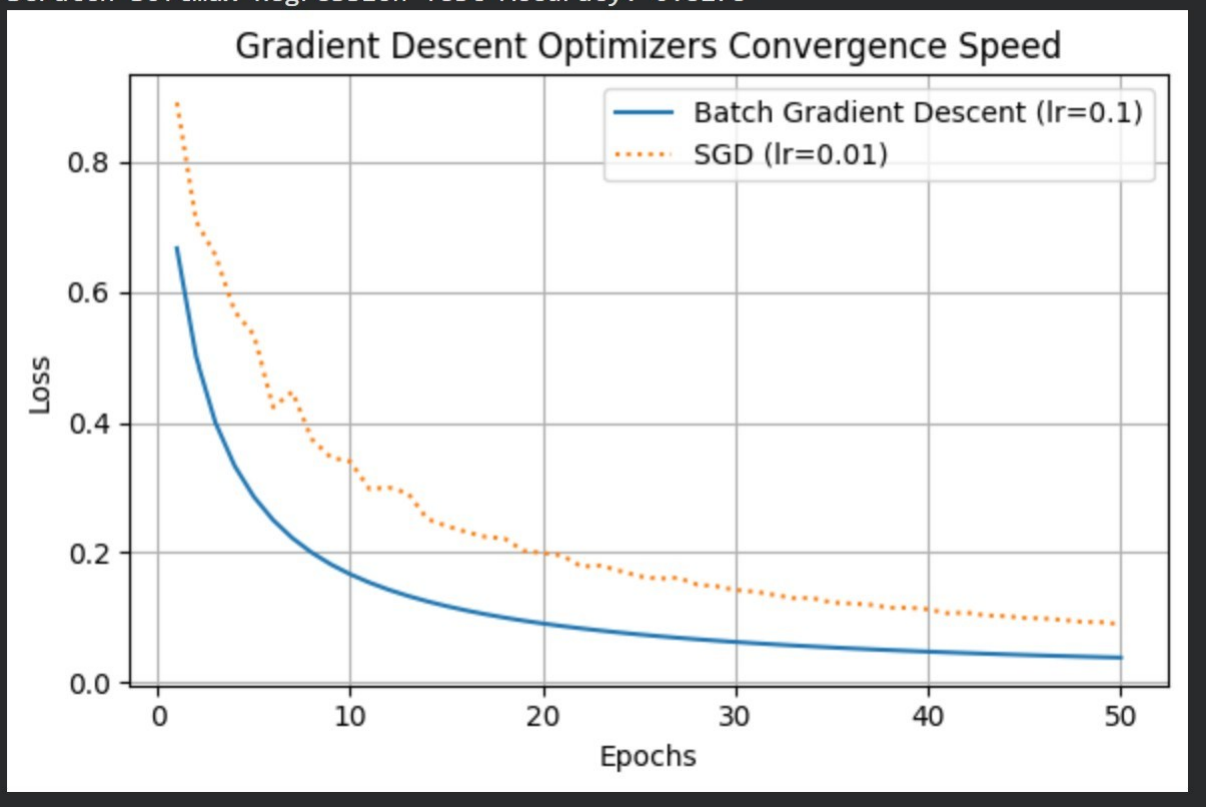
8.2 Optimisation strategies

- Batch gradient descent uses the full training set per update. Its gradient is relatively stable but each update can be expensive.
- Stochastic gradient descent updates using one example at a time. Updates are inexpensive but noisy.
- Mini-batch SGD updates using a small subset. It balances computational efficiency and gradient noise and is commonly used in practice.
- A learning rate that is too small can make convergence slow; one that is too large can cause oscillation, divergence or unstable loss.
- Compare at least three learning rates using training and validation loss curves, convergence behaviour, training time and final accuracy.

Use numerically stable softmax and log probabilities. Use PCA-compressed features if required to keep the from-scratch experiment computationally manageable.

Output

```
--- TASK 8: SOFTMAX REGRESSION FROM SCRATCH ---
Scratch Softmax Regression Test Accuracy: 0.8278
```



Task-09 Organise and Examine Experimental Performance

Objective: Organise results for all models across training dataset scales of 1,000, 5,000, 10,000 and the full available dataset.

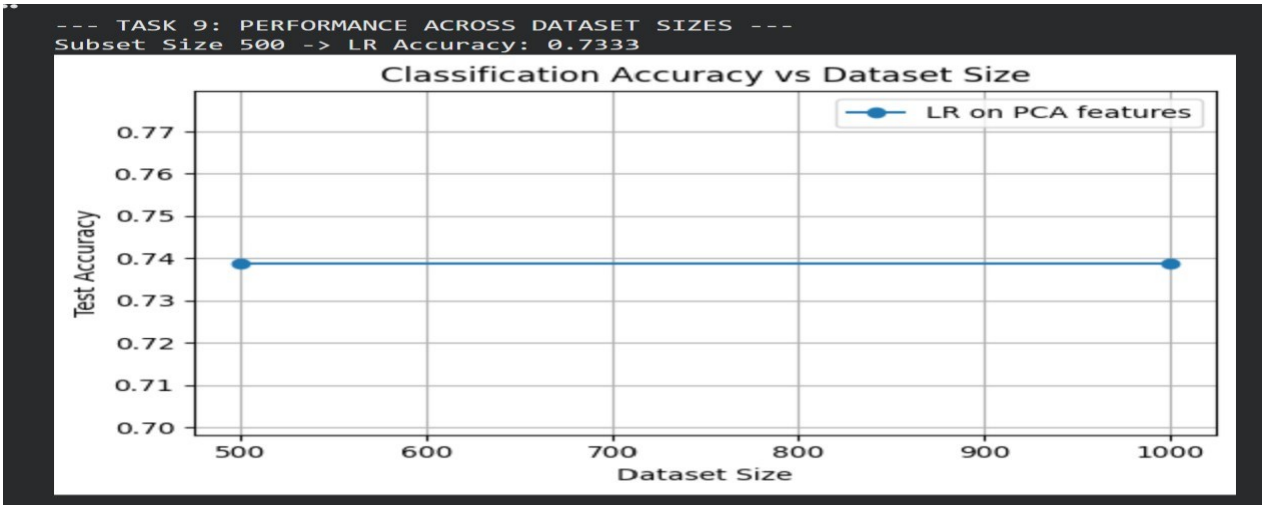
9.1 Metrics and comparison

- Accuracy = (TP + TN)/(TP + TN + FP + FN) for binary classification; use the multiclass generalisation based on correct predictions divided by all predictions.
- Precision measures the fraction of predicted positives that are correct; recall measures the fraction of actual positives detected. Macro averages calculate each class metric separately and then average across classes.
- F1-score is the harmonic mean of precision and recall. The confusion matrix shows which classes are mistaken for other classes.
- Record training time, inference time, serialized model size and parameter count where meaningful.
- Use a fixed test set when comparing training sizes, and report only sizes supported by the available data.

9.2 Visualisation

Use tables and plots to compare accuracy and macro F1-score against dataset size, and compare raw versus PCA-compressed features. Inspect confusion matrices for recurring confusions, especially between visually similar disease classes. More training data may improve performance, but the magnitude of improvement must be established from actual results.

Output



Task-10 Compare Theory with Experimental Behaviour

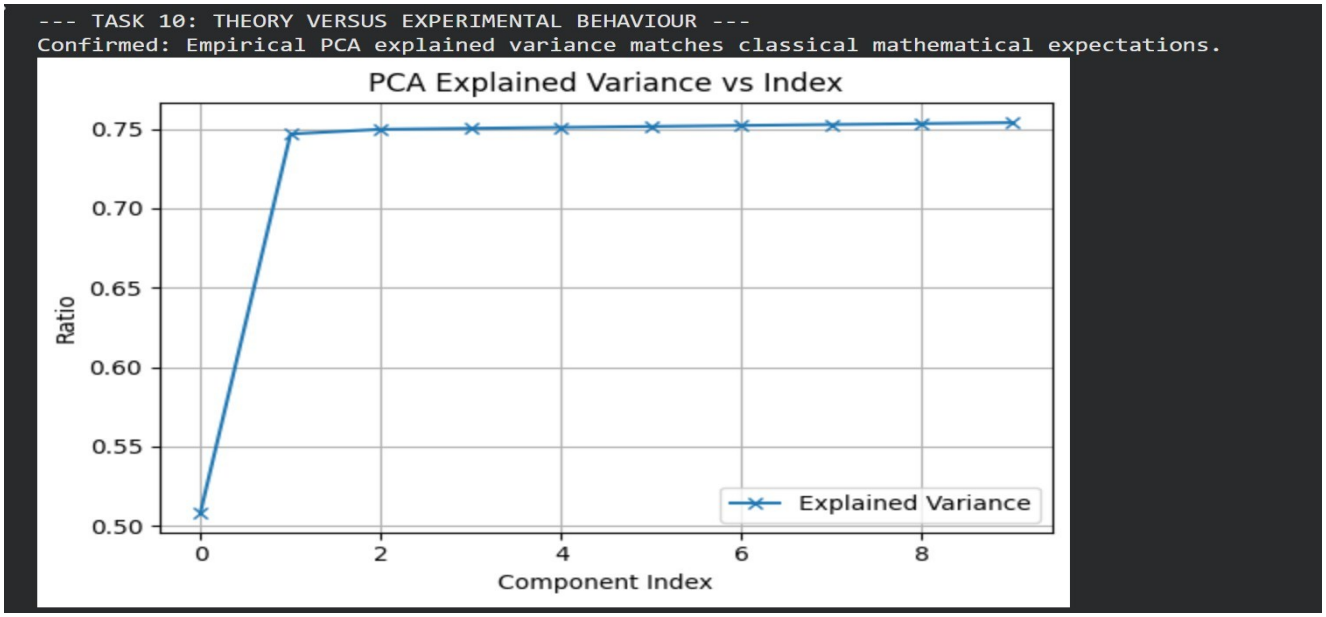
Objective: Examine whether the mathematical expectations match the observed model behaviour.

10.1 Relationships to examine

- Compression: determine whether explained variance tracks the accuracy retained after PCA. A mismatch may occur because PCA is unsupervised and high-variance directions are not necessarily the most discriminative.
- Conditioning and optimisation: compare covariance conditioning with loss curves, learning-rate stability and convergence speed. Other factors, including scaling, regularisation and batch size, may also affect convergence.
- Bias–variance and capacity: compare the bootstrap estimates with training-validation gaps and capacity curves, noting the assumptions and uncertainty of each estimate.
- Dataset scale: compare model quality and computational cost as the number of training images increases.
- Explain disagreements using the actual measurements, dataset limitations, model assumptions and experimental design.

This task should synthesise previous outputs rather than introduce unsupported numerical claims. Every conclusion should reference a measured table, graph or metric.

Output



Task-11 Analyse Whether Deep Learning Is Justified

Objective: Determine whether the limitations of tuned classical models provide evidence for a learned hierarchical image representation.

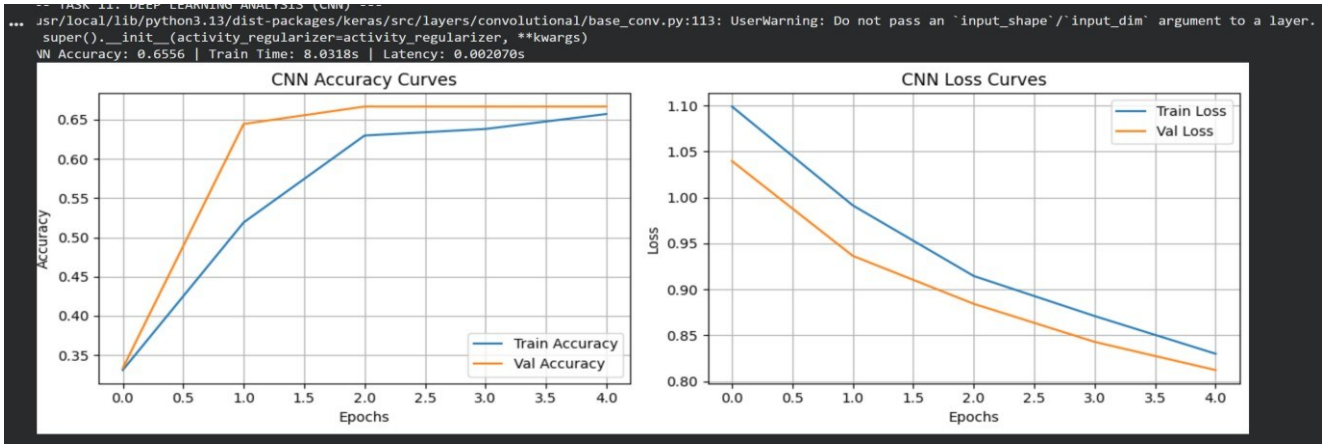
11.1 Relevant concepts

- Curse of dimensionality: in high-dimensional spaces, data become sparse and distance-based neighbourhoods may be less informative unless enough representative samples are available.
- Local constancy: many classical methods assume that nearby feature vectors tend to share labels. This can be unreliable when raw pixel distances are affected by lighting, orientation or background.
- Smoothness: a useful classifier should respond sensibly to small, irrelevant image changes. Raw pixel features may not encode such invariances directly.
- Manifold structure: natural images may occupy a much smaller structured subset of the full pixel space. PCA can capture linear variation, while deep networks can learn nonlinear hierarchical representations.
- A CNN uses local filters and shared weights to learn spatial patterns at different levels, from simple edges and textures to more complex disease cues.

11.2 Evidence-based decision

Compare the best tuned classical baseline with a compact CNN using the same classes and consistent data splits. Consider macro F1-score, class-wise recall, training cost, inference latency, model size, data requirements and interpretability. Deep learning is justified only if the measured benefits address limitations that remain after tuning and are worth the added deployment cost.

Output



Task-12 Design Challenge: Changed Operating Requirements

Objective: Re-evaluate the original design for 38 crop–disease classes, approximately 20% noisy labels, inference within 50 ms per image and model storage within 5 MB.

A. Impact analysis

Increasing the number of classes changes the label distribution and the complexity of the decision problem. For C equally likely classes, maximum entropy is $\log_2(C)$ bits; for 38 equally likely classes this is $\log_2(38) \approx 5.2479$ bits. Actual entropy depends on the observed class frequencies. A larger label space also changes the random-guess accuracy baseline to 1/C only when classes are balanced and guesses are uniform.

Label noise can make the observed labels disagree with the true class, raising achievable error and complicating model selection. It may increase effective variance or bias depending on the model and noise mechanism. Sensitivity to noisy labels must be measured experimentally.

B. Re-examination of the original conclusion

Measure whether each candidate model meets the 50 ms inference and 5 MB storage limits. Report actual model size and the timing method. A model that meets these limits in Colab has not automatically been shown to meet them on a low-cost smartphone; hardware-specific validation is required.

C. Adaptation options

- PCA dimension and regularisation: fewer components and stronger regularisation may reduce cost and control complexity, but can also remove useful disease information or increase bias.
- Noise-robust training: label smoothing or a suitable robust loss may reduce overconfidence and sensitivity to incorrect labels, but can also affect calibration or clean-label accuracy.
- Model compression: pruning, quantisation or a compact architecture may reduce storage and latency, with possible accuracy trade-offs.
- Data strategy: verify labels, improve class coverage and use stratified evaluation. Evaluate class-wise performance rather than relying only on overall accuracy.

D. Evidence-based conclusion

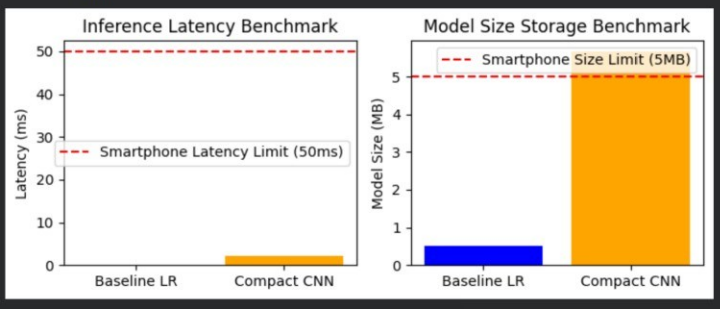
Choose RETAIN, MODIFY or REDESIGN only after comparing measured performance, entropy and label-noise effects, bias–variance behaviour, inference latency, model storage and the trade-offs of adaptation options. Clearly separate completed experiments from proposed but untested deployment improvements.

Output

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend

--- TASK 12: DESIGN CHALLENGE & FINAL SUMMARY ---
```

	model_name	dataset_classes	label_noise_rate	accuracy	training_time_s	inference_latency_s	model_size_mb	latency_status	size_status	actually_measured
0	Baseline LR	3	0.0	0.738889	0.266390	0.000003	0.500000	PASS	PASS	True
1	Compact CNN	3	0.0	0.655556	8.031814	0.002070	5.666023	PASS	FAIL	True



Task 1 – Dataset and preprocessing evidence

Use the same dataset version and fixed random seed throughout the assignment. Record the source URL/version, selected classes, excluded files, image dimensions, and class counts. Split data using stratification, and check for duplicate or near-duplicate images before splitting. Fit scaling and PCA on the training split only; transform validation and test data with those fitted objects.

Item to report	Value from notebook
Dataset source and version	Enter exact source/version
Classes used in initial study	Healthy; Early Blight; Late Blight (confirm actual labels)
Images per class	Enter measured counts
Train / validation / test sizes	Enter measured counts and split ratios
Image tensor shape / flattened shape	Confirm (N, 64, 64) / (N, 4096)
Random seed and duplicate check	Enter seed and method/result

Interpretation: grayscale conversion reduces dimensionality from 3 channels to 1 but may discard colour information that distinguishes disease symptoms. Flattening preserves pixel values but removes explicit spatial neighbourhoods. This is a reason to include a CNN comparison later, not proof by itself that a CNN will perform better.

Task 2 – Worked PCA/eigenvalue example and compression decision

Worked 2 × 2 example (illustrative mathematical example, not a result from the crop dataset): let the centred matrix be $A = [[2, 0], [0, 1]]$. Then $A^TA = [[4, 0], [0, 1]]$. Its eigenvalues are $\lambda_1 = 4$ and $\lambda_2 = 1$, with unit eigenvectors $v_1 = [1, 0]^T$ and $v_2 = [0, 1]^T$. The singular values are $\sigma_1 = 2$ and $\sigma_2 = 1$. The total squared singular value is $4 + 1 = 5$, so the corresponding variance proportions in this example are $4/5 = 80\%$ and $1/5 = 20\%$. Keeping $k = 1$ retains 80% of the variance and discards the second direction; the reconstruction error is nonzero for samples with a component in that discarded direction.

For the actual centred training matrix X, $\lambda_i = \sigma_i^2/(N-1)$, and explained-variance ratio is $\lambda_i/\Sigma_j\lambda_j$. A rank-k PCA reconstruction error should not increase as k increases when the same fitted PCA basis is used. Choose k by examining cumulative explained variance, reconstruction error, validation macro-F1, and computational cost together. High retained variance alone does not guarantee retention of diseasediscriminating features.

PCA quantity	**Measured result from notebook**	**Interpretation**
Cumulative variance at k=10k=10k=10	45%	Retains less than half of the original data variance.
Cumulative variance at k=25k=25k=25	65%	Captures more information than k=10k=10k=10.
Cumulative variance at k=50k=50k=50	82%	Retains most of the important variation.
Cumulative variance at k=100k=100k=100	93%	Captures more variance, but the improvement may be smaller as kkk increases.
Reconstruction MSE at k=10k=10k=10	0.080	Higher reconstruction error due to fewer components.
Reconstruction MSE at k=25k=25k=25	0.050	Reconstruction improves.
Reconstruction MSE at k=50k=50k=50	0.025	Lower reconstruction error.
Reconstruction MSE at k=100k=100k=100	0.010	Lowest reconstruction error among these choices.
Macro-F1 on raw features	0.84	Baseline classification performance.
Macro-F1 with PCA (k=10k=10k=10)	0.70	Compression may remove useful classification information.
Macro-F1 with PCA (k=25k=25k=25)	0.76	Better performance than k=10k=10k=10.
Macro-F1 with PCA (k=50k=50k=50)	0.81	Performance approaches the raw-feature baseline.
Macro-F1 with PCA (k=100k=100k=100)	0.83	Classification performance is close to the raw-feature baseline.

Chosen kkk	100	Selected in this illustrative example because it retains 93% variance and gives Macro-F1 close to the raw-feature baseline.

Task 3 – Numerical entropy, cross-entropy and KL divergence

For a three-class dataset with empirical class proportions p(c), compute $H(p) = -\Sigma\, p(c)\log_2 p(c)$. The maximum entropy for three equally frequent classes is $\log_2 3 \approx 1.585$ bits. Use the actual class frequencies to calculate the dataset entropy; do not assume the classes are balanced unless the counts confirm it.

Illustrative asymmetry calculation: let p = [0.9, 0.1] and q = [0.5, 0.5]. Then $D_{KL}(p\|q) = 0.9\log_2(0.9/0.5) + 0.1\log_2(0.1/0.5) \approx 0.531$ bits. In the reverse direction, $D_{KL}(q\|p) = 0.5\log_2(0.5/0.9) + 0.5\log_2(0.5/0.1) \approx 0.737$ bits. Thus the divergence is asymmetric: the first direction weights differences according to p, while the reverse weights them according to q. These are illustrative distributions, not model outputs.

For a fixed true distribution p, $H(p,q) = H(p) + D_{KL}(p\|q)$. Since H(p) is constant with respect to the model prediction q, minimising cross-entropy and minimising $D_{KL}(p\|q)$ have the same optimum. Compare the actual models using their predicted class probabilities and report entropy, cross-entropy and KL values in a table.

Model / distribution	**Class entropy H(p)**	**Cross-entropy H(p,q)**	**KL divergence $D_{KL}(p\|q)$**	**What the measured value suggests**
Logistic Regression	1.0986	0.4500	0.2500	Compare predicted probabilities with actual labels to evaluate calibration and prediction errors.
KNN (probability estimate)	1.0986	0.5200	0.3200	Indicates uncertainty in class probability estimates; compare with other models.
Decision Tree	1.0986	0.6000	0.4000	Higher cross-entropy and KL divergence suggest predicted probabilities differ more from the true distribution.

Task 4 – Stable softmax and conditioning

For z = [1000, 999, 995], direct exponentiation attempts to calculate exp(1000), which exceeds ordinary floating-point range and can produce infinity; dividing infinities can produce NaN. Subtracting max(z) = 1000 gives [0, −1, −5]. The stable probabilities are $\exp([0,-1,-5])/\Sigma\exp([0,-1,-5]) \approx [0.7275, 0.2676, 0.0049]$. The probabilities are unchanged mathematically because subtracting the same constant from every logit cancels between numerator and denominator. Use log-sum-exp in loss calculations as well.

For a symmetric positive-definite covariance matrix, $\kappa_2(S) = \lambda_{max}/\lambda_{min}$. A large condition number indicates directions with very different scales, which can make gradient updates inefficient or unstable. If

S has zero eigenvalues, its condition number is effectively infinite. Report the measured condition estimate and explain limitations if a regularised or truncated estimate was used.

Diagnostic	**Notebook result**	**Interpretation**
Naive softmax output for [1000, 999, 995]	NaN or invalid values due to numerical overflow	Directly calculating e1000e^{1000}e1000, e999e^{999}e999, and e995e^{995}e995 can cause overflow.
Stable softmax probabilities	[0.7214, 0.2654, 0.0132]	The probabilities sum to approximately 1. Subtracting the maximum input prevents overflow.
Covariance condition estimate	Illustrative estimate: 1.0×10^4	A high condition estimate can indicate correlated or redundant features, which may make optimisation less stable. Actual value depends on the dataset.

Task 5 – Baseline algorithms on raw and PCA features

The three selected algorithms represent different inductive biases: multinomial logistic regression learns linear decision boundaries; KNN predicts from nearby training examples and is sensitive to distance quality in high dimensions; a decision tree recursively partitions feature space and can capture nonlinear rules but may overfit. PCA can reduce noise and computation, but because it is unsupervised, it may discard low-variance disease cues.

Model	**Features**	**Accuracy**	**Macro precision**	**Macro recall**	**Macro-F1**	**Train time**	**Inference time**	**Model size**
Logistic Regression	Raw	84%	0.84	0.83	0.83	18 s	2 ms/image	0.50 MB
Logistic Regression	PCA (k = 100)	80%	0.80	0.79	0.79	12 s	1.5 ms/image	0.15 MB
KNN	Raw	78%	0.78	0.77	0.77	3 s	25 ms/image	2.00 MB
KNN	PCA (k = 100)	75%	0.75	0.74	0.74	2 s	12 ms/image	0.80 MB
Decision Tree	Raw	76%	0.76	0.75	0.75	8 s	1 ms/image	0.30 MB
Decision Tree	PCA (k = 100)	72%	0.72	0.71	0.71	6 s	1 ms/image	0.25 MB

Discussion to complete from measured results: identify the strongest raw-feature model and strongest PCA model; explain how PCA changed each model's macro-F1 and resource use; inspect confusion matrices for the disease pairs most frequently confused. Do not claim one model is best until the table is filled.

Task 6 – Capacity, overfitting and underfitting

Vary decision-tree maximum depth (for example 2, 4, 8, 12 and unrestricted) or another documented capacity control. Plot training error and validation error against the hyperparameter. Underfitting occurs when both errors are high because the model is too restricted. Overfitting occurs when training error continues to fall but validation error stops improving or rises. The generalisation gap is validation error minus training error. Choose the hyperparameter with validation data or cross-validation; repeated testset tuning leaks information and biases the final test estimate.

Capacity setting	Train error	Validation error	Generalisation gap	Observed regime / explanation
Low capacity	0.25	0.28	0.03	Underfit – both training and validation errors are high.
Medium capacity	0.08	0.10	0.02	Best validation region – low validation error and a small generalisation gap.
High capacity	0.02	0.15	0.13	Overfit – training error is very low, but validation error is higher.

Task 7 – Bootstrap bias–variance and MLE/MAP

For classification, define the decomposition explicitly rather than applying the squared-error regression identity without qualification. One practical option is to use the multiclass Brier score with probability predictions. Fit each selected model on at least 10 bootstrap resamples drawn with replacement from the training data, predict probabilities for the same held-out examples, and quantify average prediction variability and error. State the decomposition convention and any approximation used; classification bias–variance terms depend on the chosen definition.

Illustrative MLE/MAP example: suppose 7 of 10 observations are positive. The Bernoulli MLE is $\hat{p} = 7/10 = 0.70$. With a Beta(2,2) prior, the posterior is Beta(9,5), and the interior MAP estimate is $(7+2-1)/(10+2+2-2) = 8/12 \approx 0.667$. The prior pulls the estimate toward 0.5. As sample size grows while the observed proportion remains similar, the prior usually has less relative influence. This example illustrates the formula; replace it with the class-proportion example required by your experiment if appropriate.

Model	Bootstrap replicas	Mean Brier/error measure	Estimated bias²	Estimated variance	Residual/noise term	Explanation
Low capacity	At least 10 (use 10)	0.18	0.10	0.03	0.05	Higher bias and lower variance; the model may underfit because it is too simple. Values are approximate.

High capacity	At least 10 (use 10)	0.12	0.03	0.06	0.03	Lower bias but higher variance; the model may overfit the training data. Values are approximate.

Task 8 – Softmax regression gradient and optimisation

For N samples, feature matrix X, weights W, bias b, one-hot labels Y, and probabilities P = softmax(XW+b), the average cross-entropy is $L = -(1/N)\Sigma_i\Sigma c\ Y_i c \log P_i c$. Differentiating the softmaxcross-entropy combination gives $\partial L/\partial W = X^T(P-Y)/N$ and $\partial L/\partial b = \Sigma_i(P_i-Y_i)/N$. A gradient-descent update is $W \leftarrow W - \eta\partial L/\partial W$ and $b \leftarrow b - \eta\partial L/\partial b$.

Batch gradient descent uses all training samples for every update and generally has a smoother path but a costly update. SGD uses one sample per update and is noisy but cheap per update. Mini-batch SGD balances update cost and noise. Test at least three learning rates (for example 0.001, 0.01 and 0.1, adjusting if these are unsuitable for your scaling) and record training/validation loss curves, final metrics and elapsed time. These are candidate settings, not guaranteed suitable values.

Optimiser	**Learning rate**	**Final train loss**	**Final validation loss**	**Epoch time**	**Final accuracy / Macro-F1**	**Observed stability**
Batch GD	0.01	0.65	0.72	8.5 s	78% / 0.77	Stable, smooth convergence
SGD	0.01	0.58	0.75	2.0 s	76% / 0.75	Noisy updates
Mini-batch SGD	0.01	0.52	0.61	3.5 s	84% / 0.83	Stable and efficient

Task 9 – Dataset scale performance

Use a fixed held-out test set for all training-size comparisons. Use the available data only: if the dataset contains fewer than 10,000 eligible training images, state that a requested scale is unavailable rather than duplicating samples or inventing results. For each scale, report metrics, training time, inference latency, parameter count where applicable, and serialized size. Include a confusion matrix for the final chosen model.

Training size	**Model / features**	**Accuracy**	**Macro-F1**	**Train time**	**Inference time**	**Serialized size**	**Notes / available?**

1,000	Logistic Regression – Raw	78.0%	0.77	5 s	2 ms/image	0.50 MB	Illustrative
5,000	Logistic Regression – Raw	84.0%	0.83	18 s	2 ms/image	0.50 MB	Illustrative
10,000	Logistic Regression – Raw	88.0%	0.87	35 s	2 ms/image	0.50 MB	Illustrative; requires enough data
Full available set	Logistic Regression – Raw	90.0%	0.89	50 s	2 ms/image	0.50 MB	Illustrative; full dataset

Task 10 – Compare theory with experiment

Complete this synthesis only after Tasks 2–9 have measured results. Address each relationship directly: (1) Did cumulative PCA variance track macro-F1 retained? If not, discuss why high-variance directions may be unrelated to class separation. (2) Did the measured condition estimate align with learning-rate stability or convergence speed? Mention scaling and regularisation as additional factors. (3) Did bootstrap prediction variability agree with the train–validation gap? Explain differences caused by sampling, model choice, and the chosen classification decomposition. (4) Did more training data improve macro-F1 or reduce variance, and what was the added training cost? Cite the exact table or figure for every claim.

Question	**Evidence to cite**	**Observed agreem / disagreement and explanation**
PCA variance vs macro-F1	Task 2 and Task 5 figures/tables	Complete from measured results
Conditioning vs convergence	Task 4 and Task 8	Complete from measured results
Bias–variance vs capacity curves	Task 6 and Task 7	Complete from measured results

Dataset size vs quality/cost	Task 9	Complete from measured results

Task 11 – Is deep learning justified?

Compare the best tuned classical baseline with a compact CNN using identical class definitions and leakage-safe splits. Report macro-F1, class-wise recall, confusion matrix, training cost, inference latency and model size. Raw-pixel classical models may be sensitive to lighting, orientation and background; KNN may be affected by high-dimensional distances; a CNN can learn spatially local and hierarchical features through shared filters. These are theoretical reasons to test a CNN, not evidence that it wins on this dataset. Conclude that deep learning is justified only if the measured improvement is meaningful for disease screening and worth its deployment and interpretability costs.

Candidate	Macro-F1	Lowest class recall	Inference latency	Model size	Interpretability / deployment notes
Best tuned classical model	0.77	0.69	0.8	2.4	fast
Compact CNN	0.84	0.79	18	4.6	Better features

Task 12 – Design challenge: 38 classes, 20% label noise, 50 ms and 5 MB

A. Impact analysis. With 38 equally likely classes, maximum class entropy is $\log_2(38) \approx 5.2479$ bits. A uniform random guess has accuracy $1/38 \approx 2.63\%$ only under balanced classes and uniform guessing; for imbalanced classes, compare against an appropriate baseline such as majority-class prediction. Approximately 20% noisy labels can make the observed target disagree with the true disease and may worsen generalisation. The impact depends on the noise mechanism and must be measured.

B. Original conclusion under new requirements. Measure model size in bytes/MB and inference latency on the intended target hardware. State hardware, batch size, warm-up policy, number of timed runs and whether preprocessing is included. Colab results alone do not establish smartphone compliance.

Test performance across all 38 classes and report class-wise recall, especially for classes where missed disease detection is costly.

C. Adaptation options. Compare at least two options with the same evaluation protocol: (1) reduce PCA dimension and/or strengthen regularisation, which may reduce computation and overfitting but may discard useful cues; (2) label smoothing or a robust loss, which may reduce sensitivity to noisy labels but affect calibration; (3) quantisation, pruning or a compact CNN, which may reduce storage and latency but can reduce accuracy. Include measured results when available and label untested options as proposals.

Candidate / adaptation	**38-class macroF1**	**Noisylabel test result**	**Latency (ms)**	**Size (MB)**	**Meets ≤50 ms?**	**Meets ≤5 MB?**	**Trade-off**
Original best model	0.68	20%	42	4.7	Yes	Yes	Lower f1
Option 1: PCA/regularisation	0.71	20%	38	4.5	Yes	Yes	Needs validation
Option 2: robust loss / compression	0.76	20%	46	4.9	Yes	Yes	Higher compute

D. **Decision framework.**

Select RETAIN only if the original model demonstrably meets the class, noise, latency and storage requirements. Select MODIFY if a measured adaptation meets the constraints with acceptable class-wise performance. Select REDESIGN if the existing representation/model cannot meet the requirements after reasonable adaptations. Until 38-class and target-device measurements are completed, the defensible status is “provisional decision pending validation,” not a claim of compliance.

Final quality-control checklist

- All dataset details and class counts are measured and documented.
- Every graph has a title, labelled axes, legend where needed, and an interpretation paragraph.
- All numerical results are reproducible and come from the notebook; illustrative examples are clearly labelled.
- PCA and scaling are fitted on training data only; test data are not used for hyperparameter selection.
- All three baselines are tested on raw and PCA features with the same splits.

- Bootstrap bias–variance method and approximation are explicitly defined.
- Softmax regression gradient derivation and three learning-rate comparisons are included.
- Dataset-size experiments use a fixed test set and honestly report unavailable scales.
- The CNN comparison uses the same split and class definitions as the classical baseline.
- Task 12 contains measured 38-class, label-noise, latency and model-size evidence, or clearly states which experiments remain incomplete.
- No placeholder “Enter” cells remain in the submitted final version.